CONFIDENTIAL — PENETRATION TEST REPORT

Vulnerability Assessment & Penetration Testing

Engagement Report

Target Application: Web Application — Image File Server Endpoint (Five Lab Instances)

0a3e00060427936e80ec997400070022.web-security-academy.net ;
0aec009a031378b38269a24d00db0005.web-security-academy.net ;
0a46004203b2f0ff8255d4e200760054.web-security-academy.net ;
0aae001b0306fc0382c1977d004500a6.web-security-academy.net ;
0a37008604c8546583103c25000400d0.web-security-academy.net

Engagement ID: ENG-2026-1108-PT5

PREPARED FOR

SecureCorp — Information Security Team

PREPARED BY

Ashok — Security Analyst (VAPT)

Engagement Type	Web Application Penetration Test (Training Range)
Report Date	06 November 2026
Document Classification	Confidential — Internal / Training Use Only
Overall Risk Rating	CRITICAL
Report Version	1.0 (Final)

Document Control

Document Information

Engagement ID	ENG-2026-1108-PT5
Document Title	Vulnerability Assessment & Penetration Testing — Engagement Report
Document Owner	Ashok, Security Analyst
Classification	Confidential — shared only with named recipients on a need-to-know basis
Storage / Handling	Store encrypted at rest; transmit only over encrypted channels; destroy per client data-retention policy

Version History

Version	Date	Author	Description
0.1	06 November 2026	Ashok	Initial draft for internal technical review
1.0	06 November 2026	Ashok	Final report issued to client

Distribution List

Name	Role	Organization
Information Security Lead	Recipient	SecureCorp
Ashok	Author / Tester	Independent Security Assessor

Sign-off

Prepared by	Reviewed by	Approved by
Ashok	_____	_____
06 November 2026	Date: _____	Date: _____

Confidentiality Statement

This document and its contents are the confidential and proprietary information of the recipient organization. It has been prepared exclusively for the use of the named recipient(s) and their authorized staff, and contains sensitive technical detail — including vulnerability descriptions, exploitation steps, extracted credentials, and system configuration data — that could be used to compromise the target systems if disclosed to an unauthorized party.

This report must not be duplicated, distributed, transmitted, or disclosed, in whole or in part, to any third party without the prior written consent of the engagement sponsor. Recipients are responsible for ensuring this report is stored securely (encrypted at rest, access-controlled) and transmitted only over encrypted channels.

The findings in this report reflect the security posture of the target system(s) at the time testing was performed. No assessment can guarantee the complete absence of vulnerabilities; this report should be treated as a point-in-time assessment and supplemented with ongoing security testing and monitoring.

Testing was conducted under explicit authorization, within the scope and rules of engagement agreed upon in advance (see Section 2, Scope of Engagement). Any testing activity outside this documented scope was not performed and is not covered by this report.

Table of Contents

Document Control 2

Confidentiality Statement 3

1. Executive Summary 5

2. Scope of Engagement 8

3. Methodology 10

4. Risk Rating Methodology 13

5. Summary of Findings 14

6. Detailed Technical Findings 15

- VULN-01 — Path Traversal (Non-Recursive Filter Bypass) 15
- VULN-02 — Path Traversal (Simple Case, No Sanitization) 19
- VULN-03 — Path Traversal (Double URL-Encoding Bypass) 23
- VULN-04 — Path Traversal (Validation of Start-of-Path Bypass) 27
- VULN-05 — Path Traversal (File Extension Validation, Null Byte Bypass) 31

7. Remediation Roadmap 35

8. Conclusion & References 36

Appendix A: Evidence Index 37

Appendix B: Tools & Techniques Used 38

Appendix C: Disclaimer & Legal Notice 39

1. Executive Summary

During the course of this engagement, a penetration test was performed against five instances of a web application's image-serving endpoint, all of which retrieve image files from the server's filesystem based on a user-supplied filename parameter. The assessment followed a black-box, unauthenticated methodology, consistent with the PortSwigger Web Security Academy "File path traversal" lab series, and examined five distinct implementations of the same underlying endpoint logic — one with no input sanitization at all, one with a flawed single-pass sanitization filter, one whose literal-string traversal filter could be bypassed using double URL-encoding, one that validates only the start of the supplied path rather than its final, resolved form, and one that validates only the file extension of the supplied path without accounting for null-byte string termination.

Five (5) High severity vulnerabilities were identified, all classified as **Path Traversal** (CWE-22) in the `GET /image?filename=` endpoint of each target instance:

- **VULN-01 —** the target application attempts to sanitize the filename parameter by stripping `../` sequences, but does so only in a single, non-recursive pass. This allowed the traversal filter to be bypassed entirely using nested sequences such as `....//`, which collapse into a fresh `../` once the inner pattern is stripped.
- **VULN-02 —** a second target instance of the same endpoint logic applies **no sanitization whatsoever** to the filename parameter, allowing a direct, uncrafted `../../../etc/passwd` payload to succeed on the first attempt.
- **VULN-03 —** a third target instance blocks any request containing a literal `../` sequence, but the filter inspects the request ***before*** the final decode pass completes. Submitting a **double URL-encoded** sequence (`..%252f`) passes the filter's check and only becomes a real `../` after the pipeline's second decode pass, which runs after the filter.
- **VULN-04 —** a fourth target instance validates that the filename parameter ***starts with*** an allowed base directory (e.g. `/var/www/images/`), but never resolves or canonicalizes the full path afterward. Appending a traversal sequence after the valid prefix (`/var/www/images/../../../etc/passwd`) satisfies the prefix check while still resolving outside the intended directory at file-open time.
- **VULN-05 —** a fifth target instance validates that the requested filename ***ends with*** an allowed image extension (e.g. `.jpg` / `.png`), but a legacy, C-backed file-open layer beneath the application terminates the filename string at an embedded null byte (`%00`). Appending `%00.png` after a traversal payload satisfies the extension check at the application layer, while the OS-level file-open call discards everything from the null byte onward, opening the traversal target instead.

All five findings were fully exploited during testing. For VULN-01: a baseline request confirmed the endpoint serves files directly from the filesystem; a naive traversal payload was correctly blocked by the single-pass filter; and a crafted bypass payload (....//....//....//etc/passwd) successfully retrieved the full contents of /etc/passwd. For VULN-02: a baseline request confirmed filesystem-backed serving, and a direct, unmodified traversal payload (../../../etc/passwd) immediately returned the full contents of /etc/passwd with no filtering triggered at all. For VULN-03: a baseline request confirmed filesystem-backed serving; a naive traversal payload was correctly blocked by the literal ../ filter; and a double URL-encoded bypass payload (..%252f..%252f..%252fetc/passwd) successfully retrieved the full contents of /etc/passwd. For VULN-04: a baseline request using an absolute path within the allowed base directory succeeded, and appending a traversal sequence after that same allowed prefix (/var/www/images/../../../etc/passwd) satisfied the start-of-path check while still returning the full contents of /etc/passwd. For VULN-05: a baseline request confirmed filesystem-backed serving, and a traversal payload suffixed with a null byte and a fake allowed extension (../../../etc/passwd %00.png) satisfied the application's extension check while the underlying file-open call still opened /etc/passwd. All five findings confirm arbitrary file read outside the intended image directory, and together illustrate five distinct maturity levels of the same underlying defensive gap — absent validation, broken validation, validation

performed against the wrong (not-yet-fully-decoded) representation of the input, validation of only a prefix rather than the final, resolved path, and validation of only a suffix that a lower layer does not agree is actually the end of the string.

Severity Distribution

Severity	Count	CVSS Range
Critical	0	—
High	5	7.0 – 8.9
Medium	0	—
Low	0	—

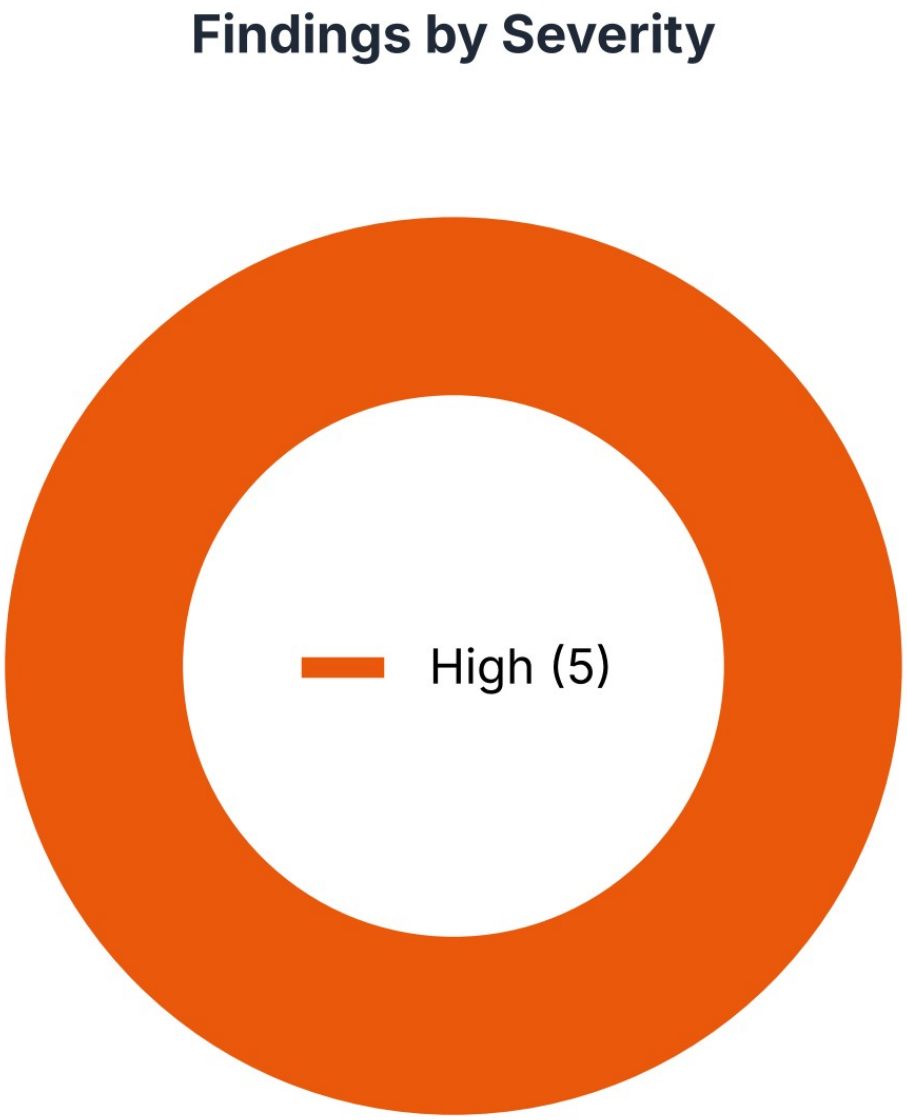


Figure A — Findings by severity (5 High).

2. Scope of Engagement

Target Information — Instance 1 (VULN-01)

Target URL	https://0a3e00060427936e80ec997400070022.web-security-academy.net/
Application	Web application serving product images via a filename-based lookup endpoint; a single-pass, non-recursive ../ filter is present
In-Scope Endpoints	GET /image?filename=
Authentication	None — publicly accessible, unauthenticated

Target Information — Instance 2 (VULN-02)

Target URL	https://0aec009a031378b38269a24d00db0005.web-security-academy.net/
Application	Web application serving product images via a filename-based lookup endpoint; no input sanitization is applied to the filename parameter
In-Scope Endpoints	GET /image?filename=
Authentication	None — publicly accessible, unauthenticated

Target Information — Instance 3 (VULN-03)

Target URL	https://0a46004203b2f0ff8255d4e200760054.web-security-academy.net/
Application	Web application serving product images via a filename-based lookup endpoint; a filter blocks literal ../ sequences but checks the request before its final URL-decode pass
In-Scope Endpoints	GET /image?filename=
Authentication	None — publicly accessible, unauthenticated

Target Information — Instance 4 (VULN-04)

Target URL	https://0aae001b0306fc0382c1977d004500a6.web-security-academy.net/
Application	Web application serving product images via a filename-based lookup endpoint; validates only that the supplied path starts with an allowed base directory, without resolving the final path
In-Scope Endpoints	GET /image?filename=
Authentication	None — publicly accessible, unauthenticated

Target Information — Instance 5 (VULN-05)

Target URL	https://0a37008604c8546583103c25000400d0.web-security-academy.net/
Application	Web application serving product images via a filename-based lookup endpoint; validates only that the supplied filename ends with an allowed image extension
In-Scope Endpoints	GET /image?filename=

Authentication	None — publicly accessible, unauthenticated

Common Engagement Details

Testing Window	06 November 2026
Tools Used	Burp Suite (Proxy/Repeater), manual payload crafting

Rules of Engagement

- Testing type: Black-box, unauthenticated, remote — conducted entirely over HTTPS against each designated lab environment.
- Testing was restricted to the five explicitly authorized targets and the single in-scope endpoint listed above on each.
- The payload used was limited to read access against /etc/passwd, a standard, non-destructive verification target for this vulnerability class; no write, modification, or destructive actions were attempted.
- All testing activity was performed within the agreed testing window using the tooling listed above.

3. Methodology

Testing was conducted in accordance with a black-box, unauthenticated methodology aligned to the Penetration Testing Execution Standard (PTES) and the OWASP Web Security Testing Guide (WSTG). The engagement followed these phases:

Phase	Activities Performed
1. Planning & Scoping	Rules of engagement, target identification, and testing window agreed with the engagement sponsor.
2. Reconnaissance	Passive and active information gathering: application fingerprinting, technology stack identification, review of client-visible source (HTML/JS) for configuration hints.
3. Scanning & Enumeration	Mapping of application functionality, input surfaces, and endpoints; identification of candidate injection points.
4. Vulnerability Analysis	Manual analysis and fault injection against identified input surfaces using Burp Suite (Proxy/Repeater) to identify exploitable weaknesses.
5. Exploitation	Controlled exploitation of confirmed vulnerabilities to demonstrate real-world impact without causing unnecessary disruption.
6. Post-Exploitation	Assessment of attainable impact (data access, code execution, privilege context) from the exploited foothold.
7. Reporting	Documentation of findings, evidence, risk ratings, and remediation guidance in this report.

Specific steps performed for the finding documented in this report are detailed in Section 6 (Detailed Technical Findings).

Finding-Specific Steps — VULN-01 (Non-Recursive Filter Bypass)

1. Baseline confirmation — requested a known-valid image (GET /image?filename=54.jpg) and confirmed the endpoint serves binary file content directly from the filesystem based on the filename parameter.
2. Naive traversal probe — submitted a standard ../../../etc/passwd payload; the request did not return the target file, indicating some form of input sanitization was in place.
3. Sanitization-logic hypothesis — inferred that the filter likely performs a single, non-recursive removal of ../ sequences, a common implementation flaw, rather than a robust allow-list or canonicalization check.
4. Bypass construction — crafted nested traversal sequences (....//) designed so that stripping an inner ../ pattern leaves a freshly reconstituted ../ behind.
5. Bypass delivery — submitted GET /image?filename=....//....//....//etc/passwd to traverse three directory levels using the reconstructed sequences.
6. Confirmation — the response returned a 200 OK with the full contents of /etc/passwd, confirming the traversal filter was fully bypassed.

Finding-Specific Steps — VULN-02 (Simple Case, No Sanitization)

7. Baseline confirmation — requested a known-valid image (GET /image?filename=7.jpg) against the second target instance and confirmed the endpoint serves binary file content directly from the filesystem.
8. Direct traversal attempt — submitted a standard, uncrafted ../../../etc/passwd payload with no modification or encoding.

9. Confirmation — the response immediately returned a 200 OK with the full contents of /etc/passwd on the first attempt, confirming the absence of any filename sanitization on this instance.

Finding-Specific Steps — VULN-03 (Double URL-Encoding Bypass)

10. Baseline confirmation — requested a known-valid image (GET /image?filename=23.jpg) against the third target instance and confirmed the endpoint serves binary file content directly from the filesystem.
11. Naive traversal probe — submitted a standard ../../../etc/passwd payload; the request was blocked, indicating a literal-string ../ filter was in place.
12. Decode-pipeline hypothesis — inferred that the filter likely inspects the request at a point before the full URL-decoding pipeline has completed, a common architectural flaw when a reverse proxy or framework layer decodes the input more than once.
13. Bypass construction — double URL-encoded the traversal sequence: each "/" was encoded to "%2f", which was then encoded a second time to "%252f", producing ..%252f..%252f..%252fetc/passwd.
14. Bypass delivery — submitted GET /image?filename=..%252f..%252f..%252fetc/passwd; the literal-../ filter found no match (the string still read "%252f" at that point) and allowed the request through.
15. Confirmation — a subsequent decode pass inside the application pipeline converted %252f → %2f → /, reconstituting ../ only after the filter had already approved the request; the response returned a 200 OK with the full contents of /etc/passwd.

Finding-Specific Steps — VULN-04 (Validation of Start-of-Path Bypass)

16. Baseline confirmation — requested a known-valid image using an absolute path within the allowed directory (GET /image?filename=/var/www/images/44.jpg) and confirmed the endpoint accepts absolute paths as long as they fall within the expected base directory.
17. Validation-logic hypothesis — inferred that the application likely checks only that the supplied string starts with the allowed base directory (e.g. a startswith()-style check), rather than resolving the full, final path before opening the file.
18. Bypass construction — appended a standard traversal sequence directly after the allowed base-directory prefix, so the string still begins with the required prefix while still containing enough ../ segments to escape it at resolution time.
19. Bypass delivery — submitted GET /image?filename=/var/www/images/../../../etc/passwd, satisfying the start-of-path check while the OS-level path resolution still walked upward out of the allowed directory.
20. Confirmation — the response returned a 200 OK with the full contents of /etc/passwd, confirming the start-of-path validation provided no real containment.

Finding-Specific Steps — VULN-05 (Null Byte Extension-Validation Bypass)

21. Baseline confirmation — requested a known-valid image (GET /image?filename=29.jpg) against the fifth target instance and confirmed the endpoint serves binary file content directly from the filesystem.
22. Validation-logic hypothesis — inferred that the application likely validates the filename by checking only that it ends with an allowed extension (e.g. endswith(".jpg")), without re-verifying the content or resolved path, and that the underlying file-open call may be backed by a legacy, null-terminated string implementation.
23. Bypass construction — appended a URL-encoded null byte (%00) followed by an allowed extension (.png) directly after a standard traversal payload, producing a string that ends in .png at the application-validation layer.

24. Bypass delivery — submitted GET /image?filename=../../../etc/passwd%00.png; the extension check saw the full string ending in .png and passed it through.
25. Confirmation — the underlying file-open call (OS/legacy-runtime level) terminated the string at the null byte, resolving the actual path to /etc/passwd and ignoring the %00.png suffix entirely; the response returned a 200 OK with the full contents of /etc/passwd, with Content-Type still reported as image/png by the application layer.

4. Risk Rating Methodology

Severity Rating Definitions

Each finding in this report is assigned a severity rating derived from its CVSS v3.1 base score, per the band definitions below (FIRST.org CVSS 3.1 Specification):

Severity	CVSS Range	Definition
Critical	9.0 – 10.0	Direct path to full system compromise; typically unauthenticated, remotely exploitable, and highly reliable. Requires immediate remediation.
High	7.0 – 8.9	Significant compromise of confidentiality, integrity, or availability; may require some precondition (authentication, user interaction) to exploit.
Medium	4.0 – 6.9	Limited impact or requires significant preconditions; still warrants remediation within a reasonable timeframe.
Low	0.1 – 3.9	Minimal direct impact; typically informational or defense-in-depth weaknesses.
Informational	0.0	Observations and best-practice recommendations with no direct, demonstrable security impact.

5. Summary of Findings

ID	Title	Severity	CVSS 3.1
VULN-01	Path Traversal via Non-Recursive Sanitization Bypass	High	7.5
VULN-02	Path Traversal — Simple Case (No Input Sanitization)	High	7.5
VULN-03	Path Traversal via Double URL-Encoding Bypass	High	7.5
VULN-04	Path Traversal via Validation of Start-of-Path Bypass	High	7.5
VULN-05	Path Traversal via File Extension Validation Null Byte Bypass	High	7.5

6. Detailed Technical Findings

VULN-01 — Path Traversal via Non-Recursive Sanitization Bypass

Finding ID	VULN-01
Severity / CVSS	High (7.5)
CWE	CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
Affected Component	GET /image?filename=
Status	Open — Not Remediated; confirmed via /etc/passwd disclosure



Figure 0 — End-to-end exploitation chain for VULN-01.

6.1 Description

The application serves image files through a GET /image?filename= endpoint that reads directly from the filesystem using the caller-supplied filename value. To prevent path traversal, the application strips ../ sequences from the filename — but performs this removal in a single pass only, without re-scanning the result. This is a classic non-recursive sanitization flaw: an attacker can construct an input containing a ../ sequence nested inside extra characters (e.g. ....//) such that, once the inner ../ is removed by the filter, the surrounding characters recombine into a brand-new ../ sequence that is never checked again. Chaining several such sequences allows an attacker to traverse an arbitrary number of parent directories and read any file accessible to the application's OS user, entirely outside the intended image directory.

6.2 Affected Component

`GET /image?filename=` — query-string parameter `filename`

6.3 Proof of Concept

Step 1 — Baseline confirmation. A known-valid filename was requested to confirm the endpoint serves files directly from the filesystem:

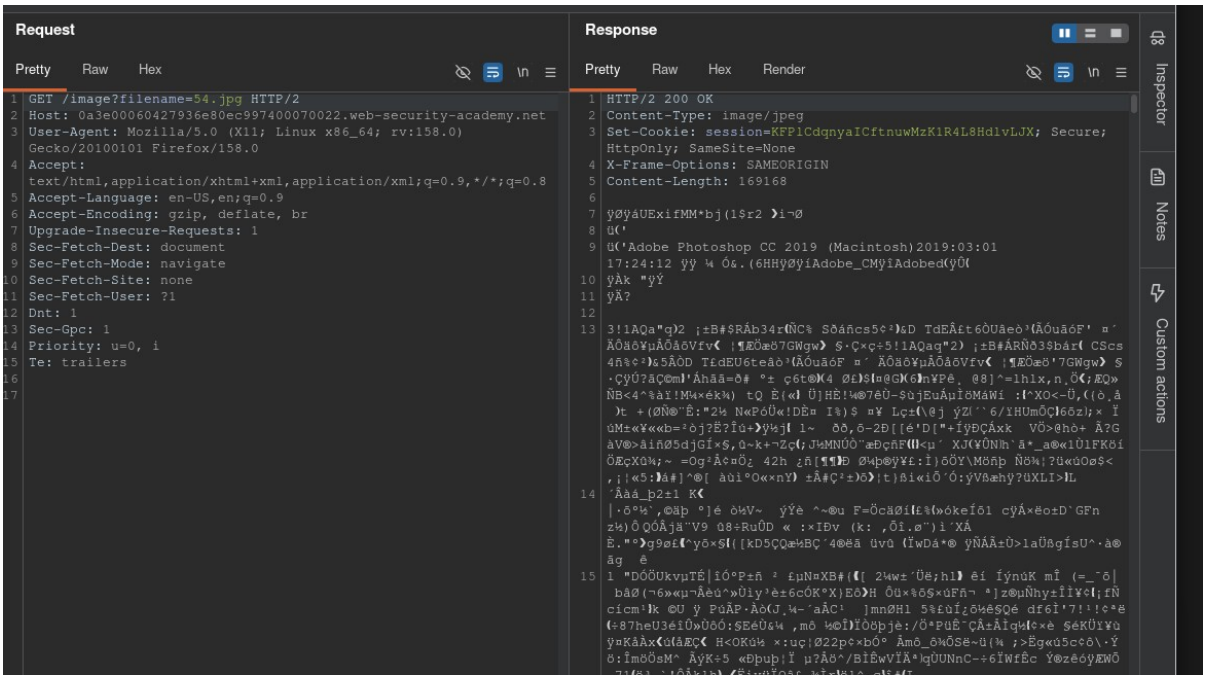


Figure 1 — Baseline request (GET /image?filename=54.jpg) returns a normal image/jpeg response, confirming filesystem-backed file serving.

Step 2 — Naive traversal attempt. A standard traversal payload was blocked, indicating the presence of some ../ sanitization:

```
GET /image?filename=../../../etc/passwd HTTP/2
(filter strips ../ sequences — request does not return /etc/passwd)
```

Step 3 — Bypass construction and delivery. Nested sequences were crafted so that a single-pass strip of the inner ../ pattern reconstitutes a fresh ../:

```
Input:    ..../
Strip "../" once (chars 2-4 = "../") leaves:
          ".." + "/"  =  "../"


Chained three times:  ....//....//....//etc/passwd
After one sanitization pass becomes:  ../../../etc/passwd
```

Reproduction:

```
GET /image?filename=....//....//....//etc/passwd HTTP/2
Host: 0a3e00060427936e80ec997400070022.web-security-academy.net
```

Result: the server responded 200 OK with Content-Type: image/jpeg (the application still believes it served an image), but the response body contained the full, unmodified contents of /etc/passwd:

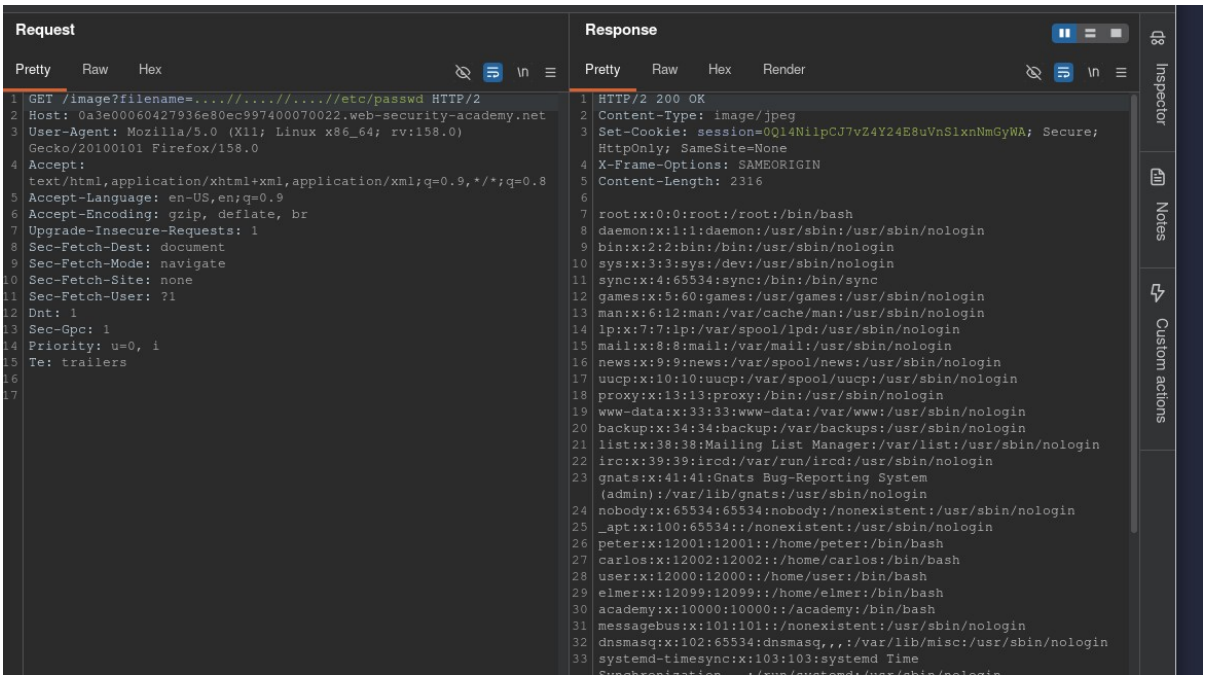


Figure 2 — Bypass payload (....//....//....//etc/passwd) returns the full contents of /etc/passwd, confirming the traversal filter is bypassed.

The disclosed file confirmed standard system accounts (root, daemon, bin, sys, etc.) alongside lab-specific application accounts (peter, carlos, user, academy), which is itself useful reconnaissance for any further authentication-focused testing against the same target.

6.4 Impact Analysis

- Confidentiality — High. Arbitrary file read is possible for any file readable by the application's OS user, including configuration files, source code, and system files such as /etc/passwd.
- Integrity — None directly (this endpoint is read-only); a write-based variant of this same flaw (e.g. in an upload or log-path parameter) would carry full integrity impact.
- Availability — None directly observed.
- Reconnaissance value — The disclosed /etc/passwd entries (including named non-system accounts) provide direct input for subsequent username enumeration or credential-attack phases.
- Blast Radius — No authentication is required; any network-reachable unauthenticated client can submit the bypass payload.

6.5 CVSS 3.1 Scoring

Vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N` **Base Score:** **7.5 (High)**

Metric	Value	Rationale
Attack Vector	Network	Exploited remotely over HTTPS against the public endpoint
Attack Complexity	Low	A single, well-documented bypass pattern; no special conditions required
Privileges Required	None	No authentication or session needed
User Interaction	None	Fully server-side; no victim action required
Confidentiality Impact	High	Arbitrary file read demonstrated (/etc/passwd)

Integrity Impact	None	Endpoint is read-only; no write primitive available here
Availability Impact	None	No denial-of-service behavior observed or attempted

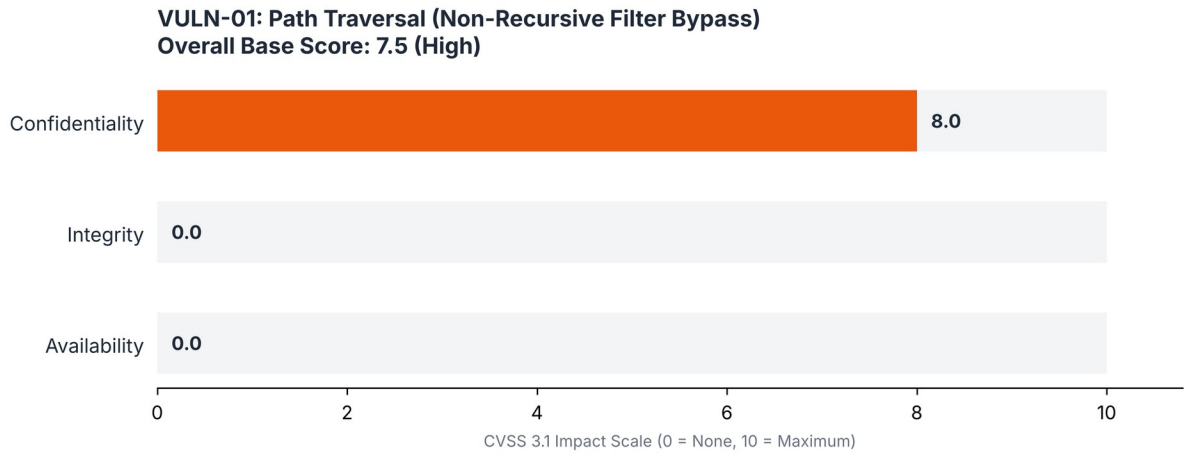


Figure B — CVSS 3.1 impact breakdown (Confidentiality / Integrity / Availability).

6.6 Remediation

1. Replace the filename parameter with an allow-list / strict format check (e.g. a regex permitting only alphanumeric characters, hyphens, underscores and a fixed set of extensions) — reject anything containing a path separator outright.
2. If arbitrary-looking filenames must be accepted, canonicalize the resolved path (os.path.realpath / Path.resolve) and verify it is still located inside the intended base directory before opening it, rather than pattern-matching on the raw input string.
3. Never rely on a single-pass string replace/strip to remove a malicious pattern — recursively re-check until no further matches are found, or avoid string-based filtering entirely in favor of the canonicalization approach above.
4. Serve files by an opaque internal identifier (e.g. a database-mapped ID) rather than by a user-supplied filesystem path, removing the path-construction step from user control entirely.
5. Apply least-privilege filesystem permissions to the application's OS user so that even a successful traversal has minimal reach beyond the intended directory.
6. Re-test with this report's bypass payload after remediation to confirm the sanitization logic correctly rejects nested and re-constituted traversal sequences.

VULN-02 — Path Traversal via Absent Input Sanitization (Simple Case)

Finding ID	VULN-02
Severity / CVSS	High (7.5)
CWE	CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
Affected Component	GET /image?filename= (second target instance)
Status	Open — Not Remediated; confirmed via /etc/passwd disclosure



Figure 0b — End-to-end exploitation chain for VULN-02.

6.7 Description

A second instance of the same image-serving endpoint logic was assessed on a separate target host. Unlike the instance covered under VULN-01, this instance applies no sanitization, filtering, or validation whatsoever to the filename parameter before using it to construct a filesystem path. As a result, a completely standard, uncrafted path-traversal payload succeeds on the first attempt, with no bypass technique required. This represents the baseline, textbook form of CWE-22: the complete absence of a defensive control, as opposed to VULN-01's presence of a flawed one.

6.8 Affected Component

`GET /image?filename=` — query-string parameter `filename` (target: 0aec009a031378b38269a24d00db0005.web-security-academy.net)

6.9 Proof of Concept

Step 1 — Baseline confirmation. A known-valid filename was requested to confirm the endpoint serves files directly from the filesystem:

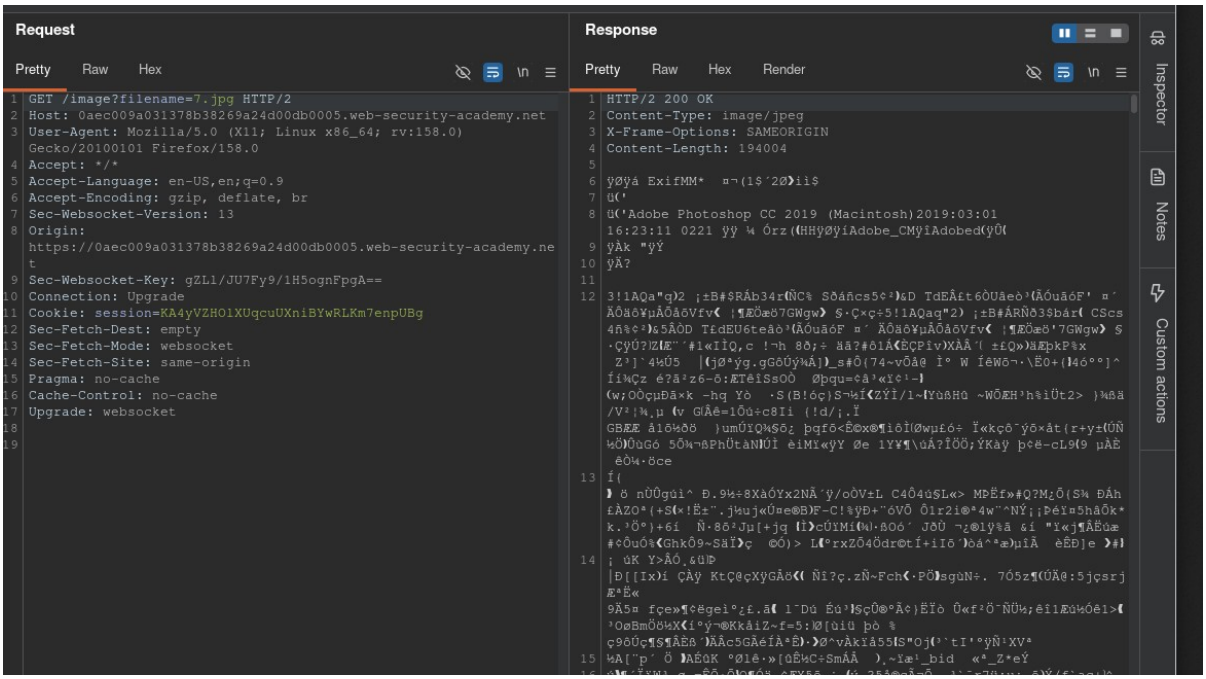


Figure 3 — Baseline request (GET /image?filename=7.jpg) returns a normal image/jpeg response, confirming filesystem-backed file serving on this instance.

Step 2 — Direct traversal. A standard, unmodified traversal payload was submitted with no encoding or crafting of any kind:

```
GET /image?filename=../../../etc/passwd HTTP/2
Host: 0aec009a031378b38269a24d00db0005.web-security-academy.net
```

Result: the server responded 200 OK, and the response body contained the full, unmodified contents of /etc/passwd on the very first attempt — confirming that no sanitization, filtering, or path-canonicalization check is applied to this parameter at all:

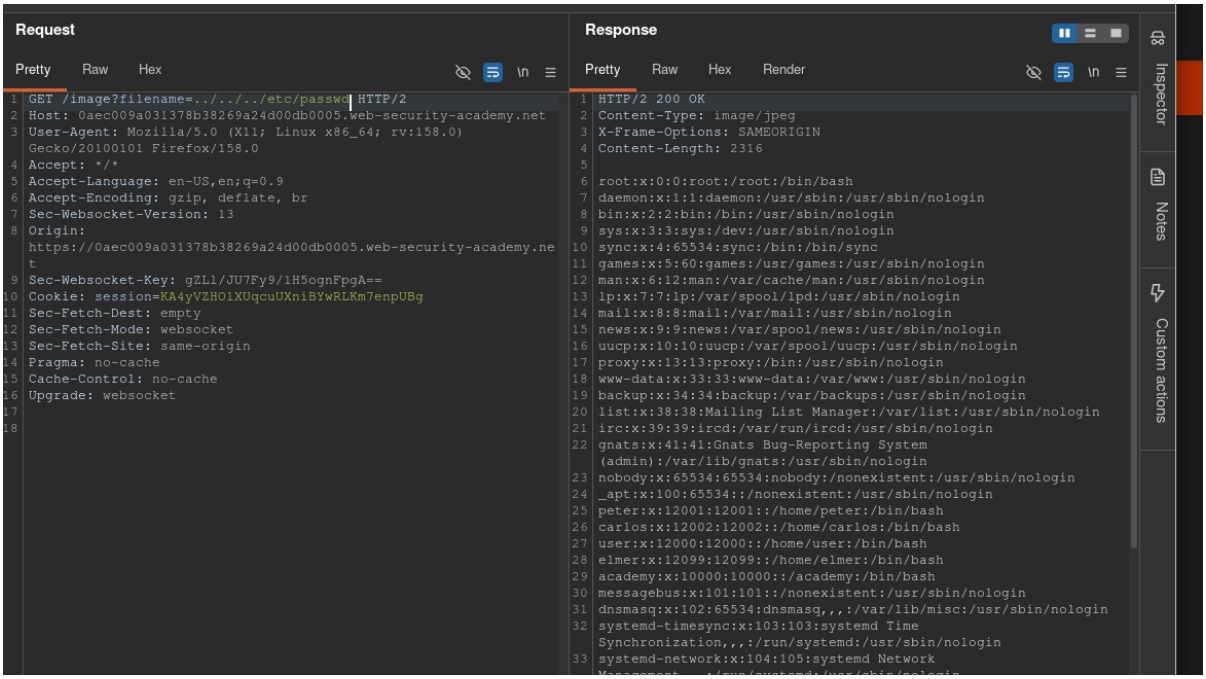


Figure 4 — Direct traversal payload (../../../etc/passwd) returns the full contents of /etc/passwd with zero filtering, confirming the complete absence of input sanitization.

No bypass crafting, encoding, or multi-step reasoning was needed for this instance — the vulnerability is exploitable by any party capable of submitting a single, unmodified HTTP request.

6.10 Impact Analysis

- Confidentiality — High. Arbitrary file read is possible for any file readable by the application's OS user, including configuration files, source code, and system files such as /etc/passwd.
- Integrity — None directly (this endpoint is read-only).
- Availability — None directly observed.
- Ease of Exploitation — Maximal. Unlike VULN-01, no understanding of internal filter logic or payload crafting is required; the standard textbook payload succeeds unmodified.
- Blast Radius — No authentication is required; any network-reachable unauthenticated client can submit the payload directly.

6.11 CVSS 3.1 Scoring

Vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N` **Base Score:** 7.5 (High)

Metric	Value	Rationale
Attack Vector	Network	Exploited remotely over HTTPS against the public endpoint
Attack Complexity	Low	No bypass crafting required at all; a standard payload succeeds unmodified
Privileges Required	None	No authentication or session needed
User Interaction	None	Fully server-side; no victim action required
Confidentiality Impact	High	Arbitrary file read demonstrated (/etc/passwd)
Integrity Impact	None	Endpoint is read-only; no write primitive available here
Availability Impact	None	No denial-of-service behavior observed or attempted

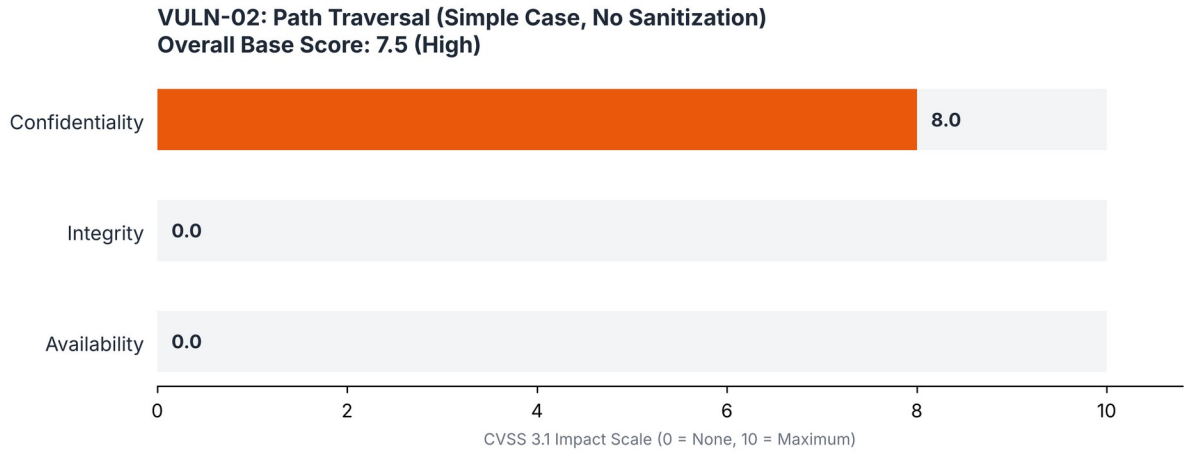


Figure C — CVSS 3.1 impact breakdown for VULN-02 (Confidentiality / Integrity / Availability).

6.12 Remediation

7. Implement input validation on the filename parameter immediately — at minimum, reject any value containing path separators (/, \) or traversal sequences (..) before this instance has even the baseline level of protection seen in VULN-01.
8. Prefer an allow-list / strict format check (e.g. alphanumeric characters, hyphens, underscores and a fixed extension set) over any blacklist-style filtering approach.
9. Canonicalize the resolved path and verify it remains inside the intended base directory before the file is opened, independent of the raw input string.
10. Serve files by an opaque internal identifier rather than by a user-supplied filesystem path, removing path construction from user control entirely.
11. Treat this finding as highest priority for remediation across the three instances, given it requires no attacker sophistication whatsoever.

VULN-03 — Path Traversal via Double URL-Encoding Bypass

Finding ID	VULN-03
Severity / CVSS	High (7.5)
CWE	CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
Affected Component	GET /image?filename= (third target instance)
Status	Open — Not Remediated; confirmed via /etc/passwd disclosure

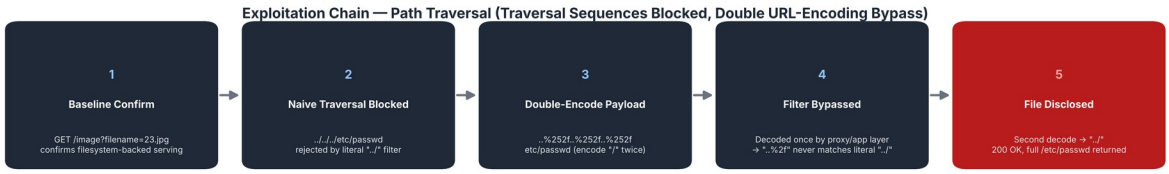


Figure 0c — End-to-end exploitation chain for VULN-03.

6.13 Description

A third instance of the same image-serving endpoint logic was assessed on a separate target host. This instance rejects any request whose filename parameter contains a literal ../ substring, which successfully blocks the naive traversal payload. However, this check is performed at a point in the request-processing pipeline before the input has been fully URL-decoded. A request layer further down the stack (the application framework's own routing/decoding logic) performs a second, independent URL-decode pass after the security check has already run. By double URL-encoding the traversal sequence — encoding "/" to "%2f", then encoding that result's "%" to "%25" to produce "%252f" — the payload passes the literal-../ filter untouched (at that point it contains no "../" substring at all), and only becomes a real ../ sequence after the pipeline's second decode pass, which happens after the filter has already approved the request.

6.14 Affected Component

`GET /image?filename=` — query-string parameter `filename` (target: 0a46004203b2f0ff8255d4e200760054.web-security-academy.net)

6.15 Proof of Concept

Step 1 — Baseline confirmation. A known-valid filename was requested to confirm the endpoint serves files directly from the filesystem:

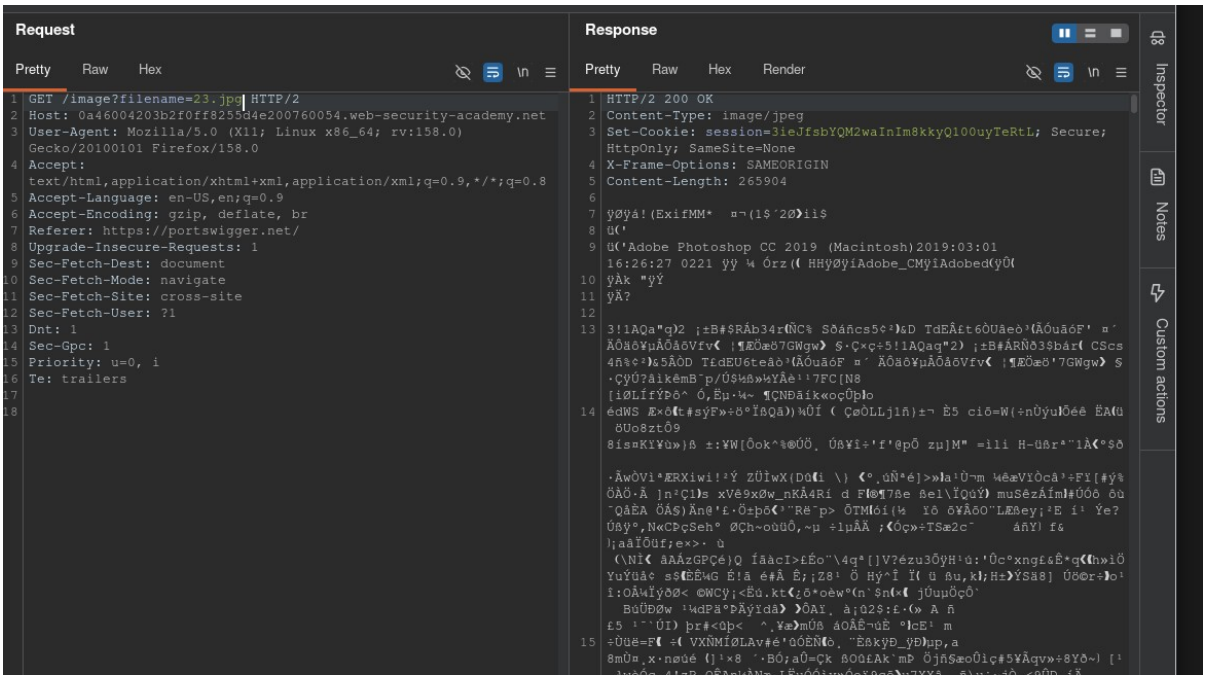


Figure 5 — Baseline request (GET /image?filename=23.jpg) returns a normal image/jpeg response, confirming filesystem-backed file serving on this instance.

Step 2 — Naive traversal probe. A standard, unmodified traversal payload was correctly blocked by the instance's literal ../ filter:

```
GET /image?filename=../../../etc/passwd HTTP/2
(request rejected — literal "../" substring detected by the filter)
```

Step 3 — Double URL-encoding the bypass. Each forward slash in the traversal sequence was URL-encoded once ("/" → "%2f"), and the resulting "%" character of that encoding was itself URL-encoded a second time ("%" → "%25"), producing a payload that never contains a literal ../ substring at the point the filter inspects it:

```
Step-by-step encoding of a single "../" segment:
  ../              (raw traversal sequence)
  ..%2f            ("/"  URL-encoded once      -> %2f)
  ..%252f          ("%"  URL-encoded a second time -> %25, giving %252f)

Chained three times:  ..%252f..%252f..%252fetc/passwd

What the filter sees (1 decode done by the time it checks):   ..%2f..%2f..%2fetc/passwd
<- no literal "../" present, PASSES the filter
What the filesystem ultimately opens (2nd decode, after the filter already
ran):  ../../../etc/passwd
```

Reproduction:

```
GET /image?filename=..%252f..%252f..%252fetc/passwd HTTP/2
Host: 0a46004203b2f0ff8255d4e200760054.web-security-academy.net
```

Result: the server responded 200 OK, and the response body contained the full, unmodified contents of /etc/passwd — confirming the literal ../ filter was bypassed by exploiting the order in which validation and decoding occur in the request pipeline:

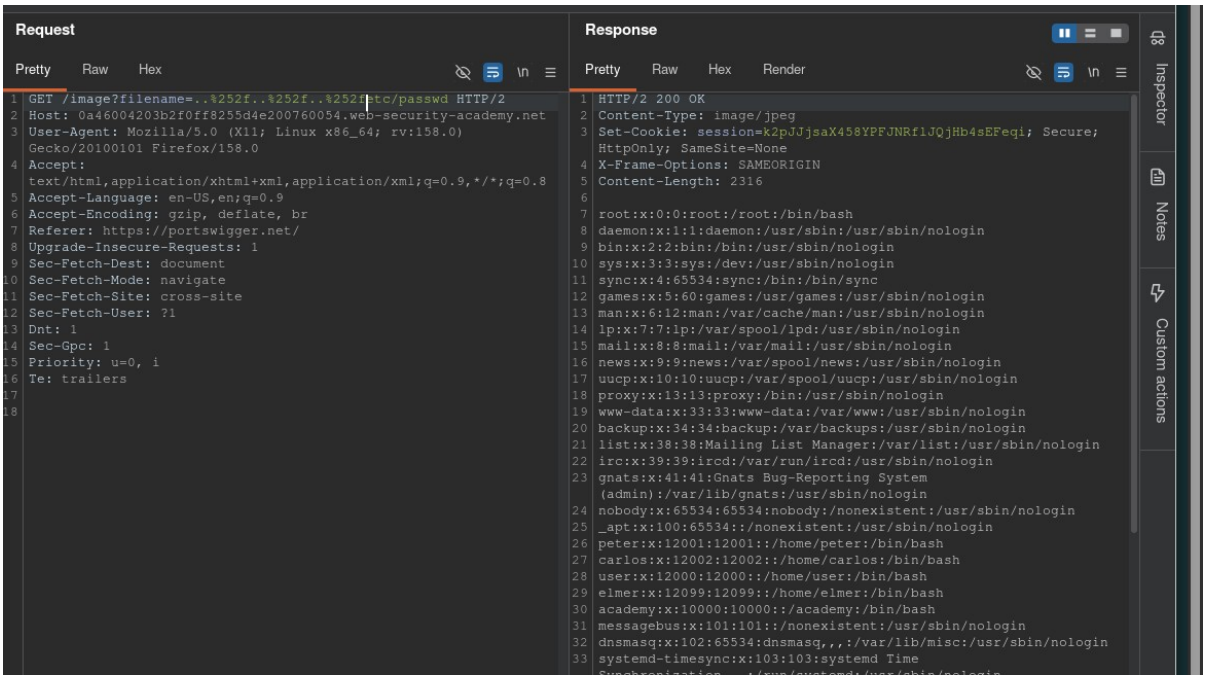


Figure 6 — Double URL-encoded payload (..%252f..%252f..%252fetc/passwd) returns the full contents of /etc/passwd, confirming the literal-../ filter is bypassed by validating an intermediate, not-yet-fully-decoded representation of the input.

This technique generalizes to any security control that validates input earlier in a multi-layer decoding pipeline than the layer that ultimately consumes it — a reverse proxy, WAF, or framework-level filter that decodes once, followed by application logic that decodes again, is a recurring source of this exact bypass class across many different vulnerability types (not only path traversal).

6.16 Impact Analysis

- Confidentiality — High. Arbitrary file read is possible for any file readable by the application's OS user, including configuration files, source code, and system files such as /etc/passwd.
- Integrity — None directly (this endpoint is read-only).
- Availability — None directly observed.
- Defense Evasion — This finding specifically defeats a literal-string security filter that was otherwise functioning as intended against the naive payload, demonstrating that the presence of a working filter for one payload form provides no assurance against an equivalent, differently-encoded form.
- Blast Radius — No authentication is required; any network-reachable unauthenticated client capable of URL-encoding a string twice can submit the bypass payload.

6.17 CVSS 3.1 Scoring

Vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N` **Base Score:** **7.5 (High)**

Metric	Value	Rationale
Attack Vector	Network	Exploited remotely over HTTPS against the public endpoint
Attack Complexity	Low	A well-documented double-encoding technique; no special conditions required
Privileges Required	None	No authentication or session needed
User Interaction	None	Fully server-side; no victim action required

Confidentiality Impact	High	Arbitrary file read demonstrated (/etc/passwd)
Integrity Impact	None	Endpoint is read-only; no write primitive available here
Availability Impact	None	No denial-of-service behavior observed or attempted

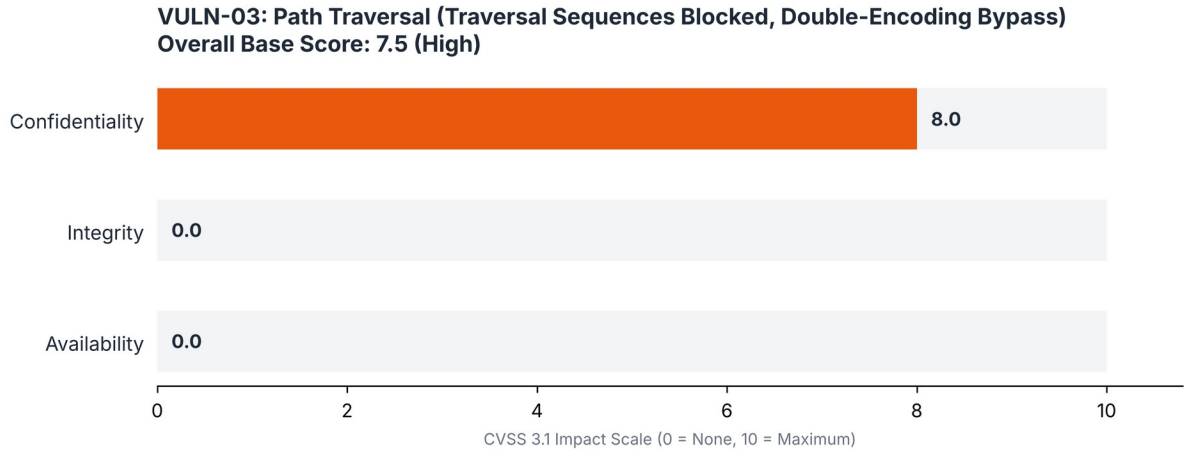


Figure D — CVSS 3.1 impact breakdown for VULN-03 (Confidentiality / Integrity / Availability).

6.18 Remediation

12. Perform all security-relevant input validation on the fully and finally decoded value of the input — never on an intermediate representation that a later layer will decode further.
13. Where multiple layers (reverse proxy, WAF, application framework) each perform their own URL-decoding, ensure decoding happens exactly once, at a single well-defined point, before any validation runs.
14. Reject any request whose filename parameter contains percent-encoded characters at all, unless a specific, justified use case requires them — most legitimate filenames never need URL-encoded content.
15. Replace literal-string / blacklist-style traversal filters with allow-list validation or canonicalize-then-verify logic (as recommended for VULN-01 and VULN-02); these approaches are immune to encoding-order bypasses because they operate on the final, resolved filesystem path rather than on the raw string.
16. Re-test with this report's double URL-encoded payload after remediation to confirm the fix validates the fully-decoded value and not an intermediate form.

VULN-04 — Path Traversal via Validation of Start-of-Path Bypass

Finding ID	VULN-04
Severity / CVSS	High (7.5)
CWE	CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
Affected Component	GET /image?filename= (fourth target instance)
Status	Open — Not Remediated; confirmed via /etc/passwd disclosure



Figure 0d — End-to-end exploitation chain for VULN-04.

6.19 Description

A fourth instance of the same image-serving endpoint logic was assessed on a separate target host. This instance accepts absolute filesystem paths for the filename parameter, and validates them by checking that the supplied string begins with an allowed base directory (e.g. /var/www/images/) — a common pattern intended to restrict file access to a single directory. However, this check operates purely at the string level: it confirms the start of the string, but never resolves or canonicalizes the full path before the file is opened. An attacker can satisfy the prefix check by starting the payload with the exact allowed directory string, and then append a standard ../ traversal sequence immediately after it. The string-level check passes (the payload genuinely starts with /var/www/images/), but the filesystem's own path resolution logic, which runs afterward at open() time, walks back up through each ../ segment and exits the allowed directory entirely.

6.20 Affected Component

`GET /image?filename=` — query-string parameter `filename` (target: 0aae001b0306fc0382c1977d004500a6.web-security-academy.net)

6.21 Proof of Concept

Step 1 — Baseline confirmation. A known-valid absolute path within the allowed base directory was requested, confirming the endpoint accepts absolute paths as long as they fall inside the expected directory:

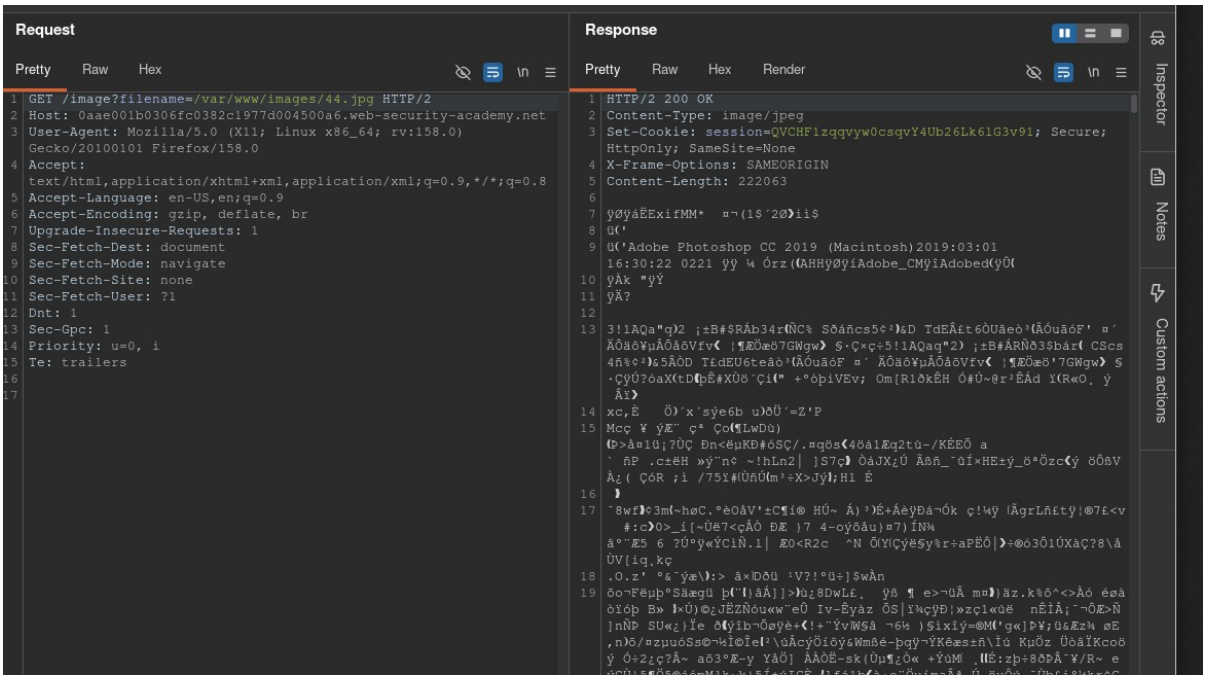


Figure 7 — Baseline request (GET /image?filename=/var/www/images/44.jpg) returns a normal image/jpeg response, confirming the endpoint accepts absolute paths within the allowed base directory.

Step 2 — Bypass construction. A standard traversal sequence was appended directly after the same allowed base-directory prefix, so the resulting string still begins with the required prefix while still containing enough ../ segments to walk back out of it once the OS resolves the path:

```
Allowed prefix (passes the startswith() check):   /var/www/images/
Appended traversal:                               ../../../etc/passwd

Combined payload:   /var/www/images/../../../etc/passwd

String-level check:  startswith("/var/www/images/")  ->  TRUE  (check passes)

Filesystem resolution at open() time (runs AFTER the check, not checked again):
  /var/www/images/../../../etc/passwd
  ->  /var/www/../../etc/passwd
  ->  /var/../etc/passwd
  ->  /etc/passwd
```

Reproduction:

```
GET /image?filename=/var/www/images/../../../etc/passwd HTTP/2
Host: 0aae001b0306fc0382c1977d004500a6.web-security-academy.net
```

Result: the server responded 200 OK, and the response body contained the full, unmodified contents of /etc/passwd — confirming that the start-of-path validation provided no real path containment, since it never inspected the fully-resolved destination of the request:

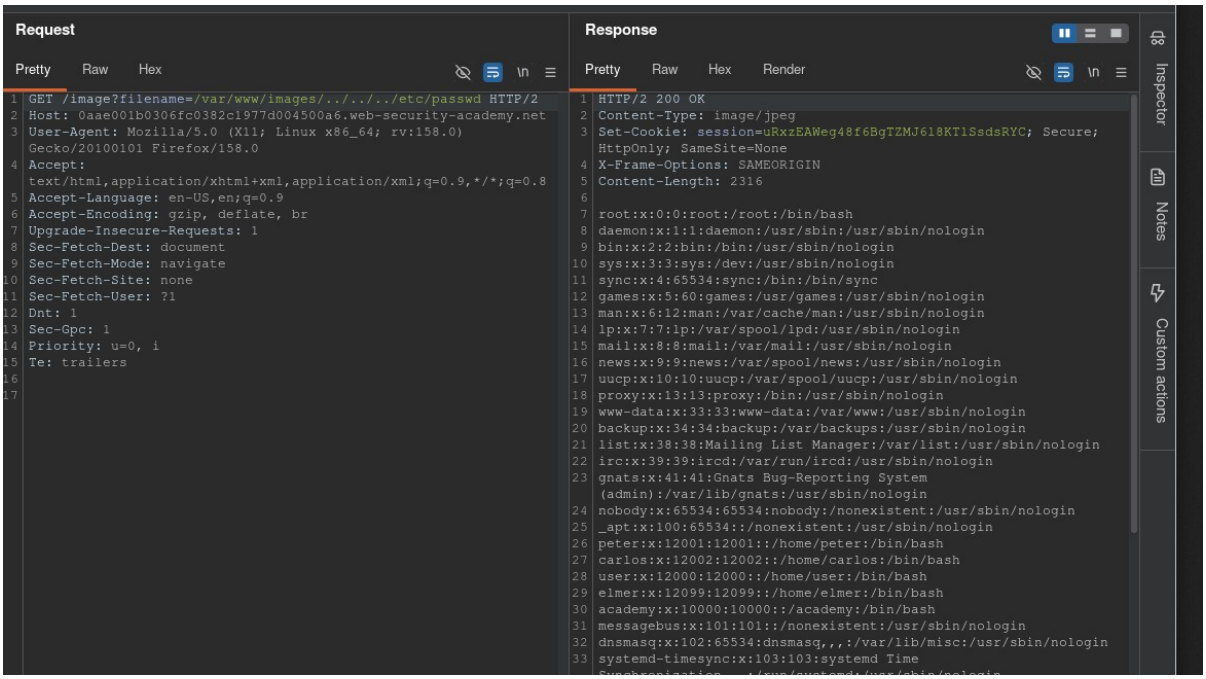


Figure 8 — Prefix-plus-traversal payload (/var/www/images/../../../etc/passwd) returns the full contents of /etc/passwd, confirming the start-of-path check is not equivalent to a containment check.

This finding is a close structural relative of VULN-01 and VULN-03: in all three cases, the application performs a check against one representation of the input (a stripped string, a partially-decoded string, or a string prefix) while the filesystem ultimately acts on a different, fully-resolved representation that the check never examined.

6.22 Impact Analysis

- Confidentiality — High. Arbitrary file read is possible for any file readable by the application's OS user, including configuration files, source code, and system files such as /etc/passwd.
- Integrity — None directly (this endpoint is read-only).
- Availability — None directly observed.
- Containment Failure — The directory-restriction control gives the appearance of sandboxing file access to a single folder, but provides none in practice; this is a particularly deceptive control for developers to rely on.
- Blast Radius — No authentication is required; any network-reachable unauthenticated client can submit the payload directly, with only the known or guessable base-directory name as a prerequisite.

6.23 CVSS 3.1 Scoring

Vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N` **Base Score:** 7.5 (High)

Metric	Value	Rationale
Attack Vector	Network	Exploited remotely over HTTPS against the public endpoint
Attack Complexity	Low	Only requires knowing/guessing the allowed base directory and appending a standard traversal sequence
Privileges Required	None	No authentication or session needed
User Interaction	None	Fully server-side; no victim action required

Confidentiality Impact	High	Arbitrary file read demonstrated (/etc/passwd)
Integrity Impact	None	Endpoint is read-only; no write primitive available here
Availability Impact	None	No denial-of-service behavior observed or attempted

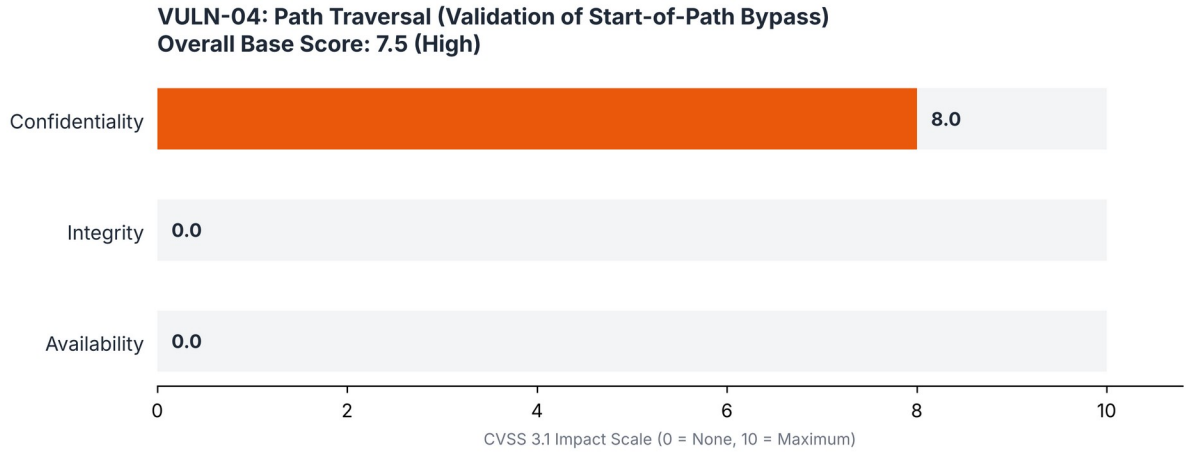


Figure E — CVSS 3.1 impact breakdown for VULN-04 (Confidentiality / Integrity / Availability).

6.24 Remediation

17. Never validate a path by checking only its prefix (startswith()); always resolve the path to its final, canonical, absolute form first (os.path.realpath() / Path.resolve()), then verify that the resolved result is located inside the allowed base directory.
18. Compare resolved paths using a proper containment check (e.g. os.path.commonpath([base, resolved]) == base), not a substring or prefix comparison on the raw, unresolved input.
19. Reject any filename parameter containing traversal sequences (../) outright, as a defense-in-depth measure in addition to canonicalize-then-verify, rather than relying on either control alone.
20. Serve files by an opaque internal identifier rather than by a user-supplied filesystem path, removing path construction from user control entirely — the most robust fix across all four findings in this report.
21. Re-test with this report's prefix-plus-traversal payload after remediation to confirm the fix resolves the path before validating containment.

VULN-05 — Path Traversal via File Extension Validation Null Byte Bypass

Finding ID	VULN-05
Severity / CVSS	High (7.5)
CWE	CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'); related CWE-626 (Null Byte Interaction Error)
Affected Component	GET /image?filename= (fifth target instance)
Status	Open — Not Remediated; confirmed via /etc/passwd disclosure



Figure 0e — End-to-end exploitation chain for VULN-05.

6.25 Description

A fifth instance of the same image-serving endpoint logic was assessed on a separate target host. This instance allows traversal sequences to pass through unfiltered, but relies on a file-extension allow-list as its containment control: it checks that the supplied filename ends with an approved image extension (e.g. .jpg or .png) before serving the file, on the assumption that a string ending in an image extension cannot meaningfully reference an arbitrary system file. This assumption breaks down because the extension-check code and the underlying file-open call do not agree on where the string ends. By embedding a URL-encoded null byte (%00) between the traversal path and a fake trailing extension, the extension-check layer sees and approves the full string (which genuinely ends in .png), while the lower-level file-open call — backed by null-terminated string semantics — stops reading at the null byte and opens only the portion before it, which is the traversal target.

6.26 Affected Component

`GET /image?filename=` — query-string parameter `filename` (target: 0a37008604c8546583103c25000400d0.web-security-academy.net)

6.27 Proof of Concept

Step 1 — Baseline confirmation. A known-valid filename was requested to confirm the endpoint serves files directly from the filesystem:

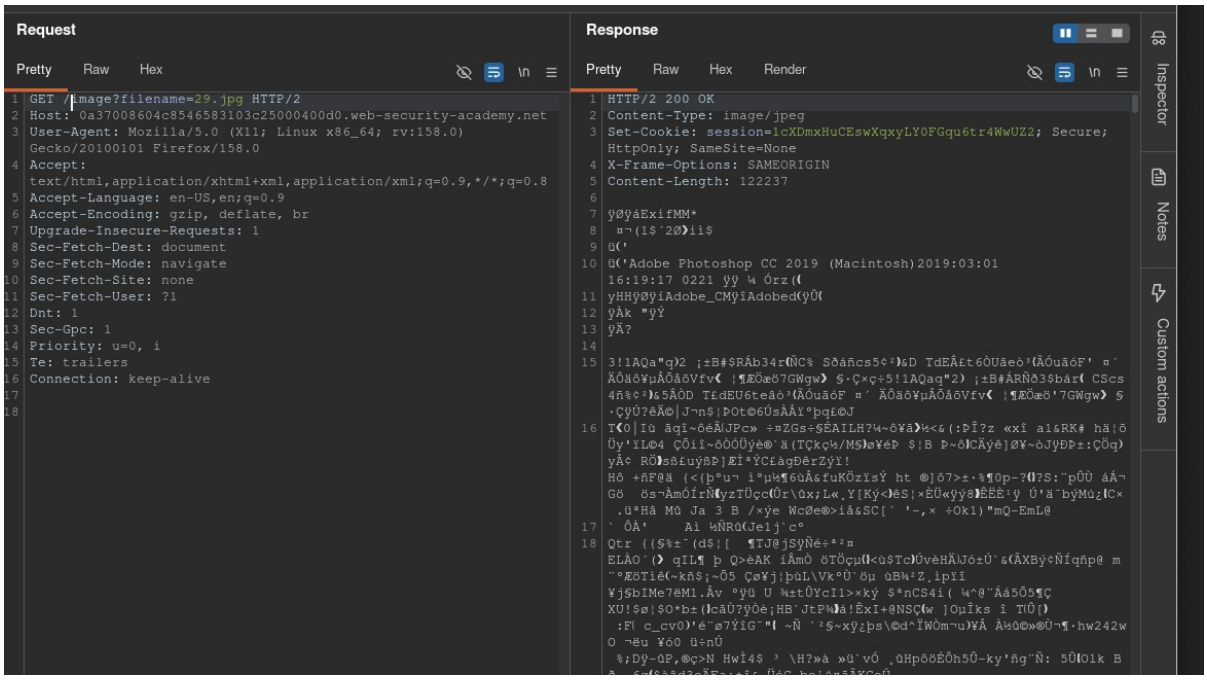


Figure 9 — Baseline request (GET /image?filename=29.jpg) returns a normal image/jpeg response, confirming filesystem-backed file serving on this instance.

Step 2 — Bypass construction. A standard traversal payload was suffixed with a URL-encoded null byte and a fake allowed extension, so the string satisfies the extension check while the real file-open call still terminates at the null byte:

```
Traversal target:            ../../../etc/passwd
Null byte (URL-encoded):     %00
Fake trailing extension:     .png

Combined payload:    ../../../etc/passwd%00.png

Application extension check (sees the FULL string):
  "...etc/passwd%00.png".endswith(".png")  ->  TRUE  (check passes)

File-open call (null-terminated string semantics, runs AFTER the check):
  "...etc/passwd\x00.png"
  ->  string reading STOPS at \x00
  ->  actual path opened:  "...etc/passwd"
```

Reproduction:

```
GET /image?filename=../../../etc/passwd%00.png HTTP/2
Host: 0a37008604c8546583103c25000400d0.web-security-academy.net
```

Result: the server responded 200 OK with Content-Type: image/png (reflecting the application layer's belief that a .png file was served), but the response body contained the full, unmodified contents of /etc/passwd — confirming that the extension-validation layer and the file-serving layer disagreed about where the filename string actually ended:

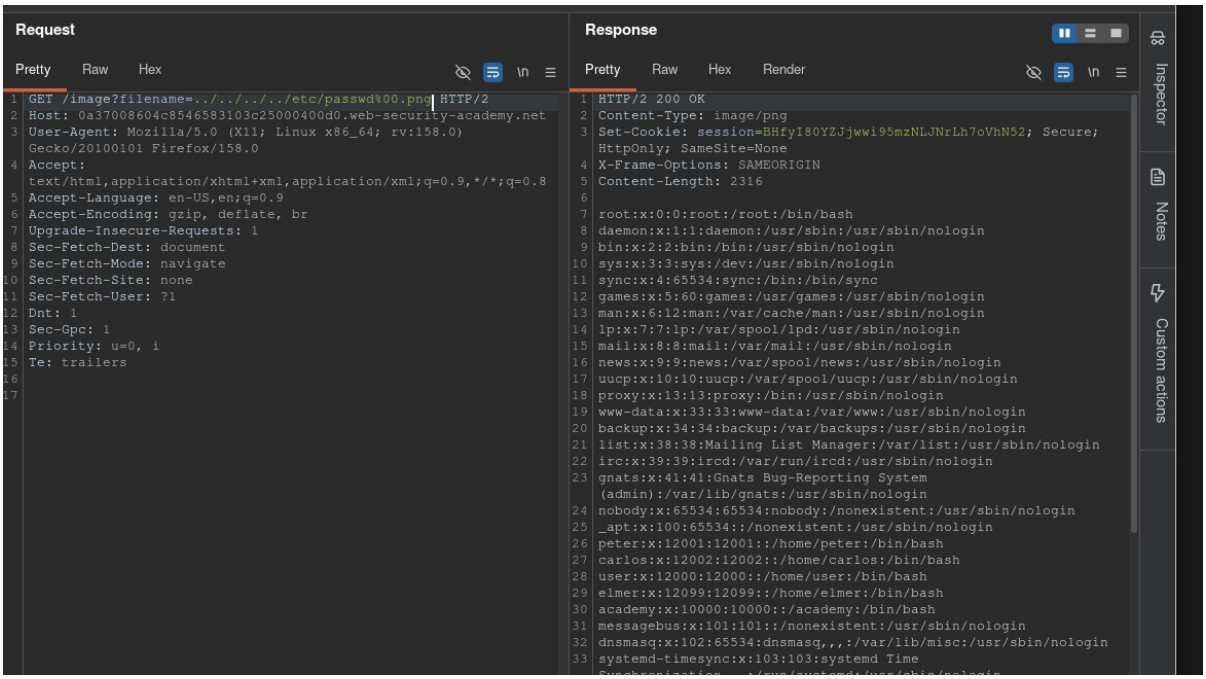


Figure 10 — Null-byte payload (../../../etc/passwd%00.png) returns the full contents of /etc/passwd despite a Content-Type: image/png header, confirming the extension check is bypassed via null-byte string truncation.

This technique is a historical/legacy-runtime-dependent bug (classically seen in PHP ≤5.3 and C-backed file APIs); modern language runtimes increasingly reject or safely handle embedded null bytes by default. Its presence here nonetheless confirms a legacy or native code path is in use for file resolution on this instance, and it remains an important pattern to test for in any codebase with older dependencies, native extensions, or C-library file-handling calls.

6.28 Impact Analysis

- Confidentiality — High. Arbitrary file read is possible for any file readable by the application's OS user, including configuration files, source code, and system files such as /etc/passwd.
- Integrity — None directly (this endpoint is read-only).
- Availability — None directly observed.
- Containment Failure — The extension allow-list gives the appearance of restricting the endpoint to image files only, but provides no real guarantee once the file-open layer's string-termination behavior diverges from the validation layer's.
- Blast Radius — No authentication is required; any network-reachable unauthenticated client capable of URL-encoding a null byte can submit the bypass payload.

6.29 CVSS 3.1 Scoring

Vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N` **Base Score:** **7.5 (High)**

Metric	Value	Rationale
Attack Vector	Network	Exploited remotely over HTTPS against the public endpoint
Attack Complexity	Low	A well-documented null-byte suffix technique; no special conditions required beyond a legacy-compatible file-open layer
Privileges Required	None	No authentication or session needed

User Interaction	None	Fully server-side; no victim action required
Confidentiality Impact	High	Arbitrary file read demonstrated (/etc/passwd)
Integrity Impact	None	Endpoint is read-only; no write primitive available here
Availability Impact	None	No denial-of-service behavior observed or attempted

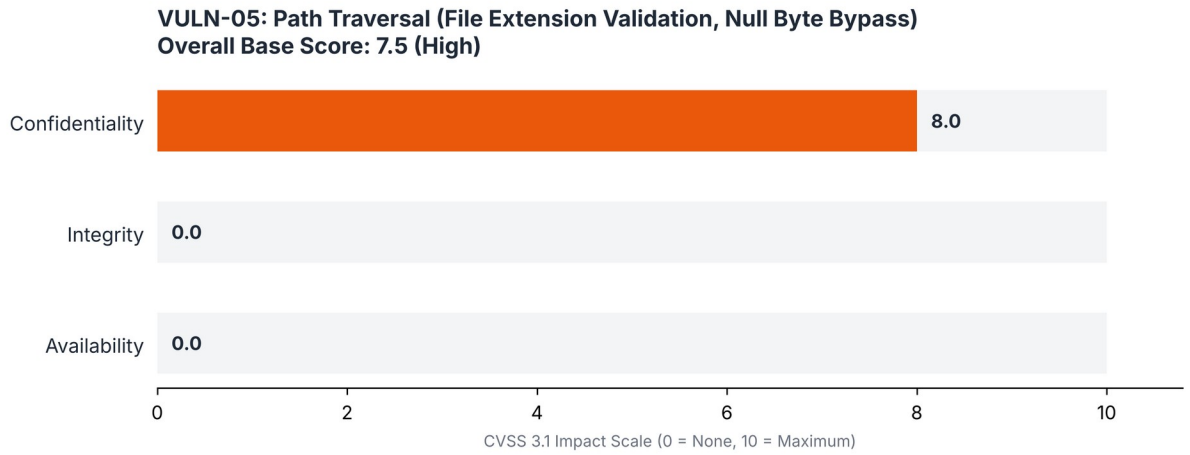


Figure F — CVSS 3.1 impact breakdown for VULN-05 (Confidentiality / Integrity / Availability).

6.30 Remediation

22. Never rely on extension-suffix checks (endswith()) as a security boundary; validate the fully-resolved, canonical file path and its containment within the allowed base directory instead.
23. Reject any filename parameter containing embedded null bytes or other control characters outright, as a defense-in-depth measure, regardless of the language runtime's current null-byte handling behavior.
24. Where legacy or native/C-backed file-handling code paths are unavoidable, ensure a modern, null-byte-aware validation layer runs immediately before them and operates on the exact same string the native call will consume.
25. Serve files by an opaque internal identifier rather than by a user-supplied filesystem path, removing path construction from user control entirely — the most robust fix across all five findings in this report.
26. Re-test with this report's null-byte payload after remediation to confirm the fix rejects or safely normalizes embedded null bytes before any file-system interaction occurs.

7. Remediation Roadmap

The following prioritized action plan is recommended based on the severity and exploitability of the findings in this report:

Priority	Target Timeframe	Action
P1 — Critical Priority	Immediate (0–3 days)	VULN-02: implement baseline input validation on the second instance's filename parameter immediately — currently zero defense is applied, making this the highest-priority item across all five findings.
P1 — High	Immediate (0–7 days)	VULN-01, VULN-03, VULN-04 and VULN-05: replace each instance's filename validation with canonicalize-then-verify logic that resolves the path to its final absolute form (rejecting embedded null bytes outright) and checks containment against the allowed base directory — this single fix pattern resolves the non-recursive-strip, double-decode-order, prefix-only-check, and extension-suffix-check flaws together.
P2 — Medium	30 days	On all five instances: migrate to serving files by opaque internal ID rather than user-supplied filename; apply least-privilege filesystem permissions to the serving process; ensure URL-decoding happens exactly once, at a single well-defined point, before any validation; reject embedded null bytes and control characters across all input-handling code.
P3 — Low	90 days	Audit the codebase for other single-pass, absent, decode-order-dependent, prefix-only, or suffix-only string-validation patterns (XSS filters, SQL-escaping helpers, WAF rules, access-control checks, upload-extension checks, etc.) that could share these same logical flaws.

8. Conclusion & References

Five instances of the same image-serving endpoint logic were assessed, all found vulnerable to Path Traversal (CWE-22), and all five exploited end-to-end during this engagement to read /etc/passwd from the underlying host. On the first instance (VULN-01), the filename parameter's path-traversal sanitization applies a single, non-recursive string replacement, which can be fully bypassed by nesting ../ inside extra characters (..../) that reconstitute a valid traversal sequence once the inner pattern is removed. On the second instance (VULN-02), no sanitization is applied at all, and a completely standard, uncrafted ../../../etc/passwd payload succeeds on the first attempt. On the third instance (VULN-03), a literal-string ../ filter is bypassed by double URL-encoding the traversal sequence (..%252f), which passes the filter's check and only resolves into a real ../ after a second decode pass that runs later in the request pipeline. On the fourth instance (VULN-04), a start-of-path check validates only that the supplied string begins with an allowed base directory, allowing a traversal sequence appended directly after that prefix to satisfy the check while still resolving outside the directory at file-open time. On the fifth instance (VULN-05), an extension allow-list validates only that the supplied string ends with an approved image extension, while a null byte embedded before that extension causes the underlying file-open call to terminate the string early and open the traversal target instead. Together these five findings illustrate a spectrum of defensive maturity — the complete absence of a control, a control that only appears to work, a control that validates the wrong representation of its own input, a control that inspects only a fragment (prefix or suffix) of the input rather than its final resolved form, and the recurring meta-lesson that validation and consumption must always agree on how a string is parsed. Given the complete absence of authentication on all five instances and the low-complexity, well-documented nature of all five exploitation paths, remediation is recommended at High priority for VULN-01, VULN-03, VULN-04 and VULN-05, and at Critical priority for VULN-02, given the latter requires no attacker sophistication whatsoever.

References

- CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') — https://cwe.mitre.org/data/definitions/22.html
- CWE-626: Null Byte Interaction Error (Poison Null Byte) — https://cwe.mitre.org/data/definitions/626.html
- OWASP Web Security Testing Guide — Testing for Path Traversal (WSTG-ATHZ-01) — https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/01-Testing_Directory_Traversal_File_Include
- PortSwigger Web Security Academy — Directory traversal (lab series, including non-recursive stripping, encoding-bypass, start-of-path-validation, and null-byte-extension-bypass variants) — https://portswigger.net/web-security/file-path-traversal
- OWASP — Path Traversal attack reference — https://owasp.org/www-community/attacks/Path_Traversal
- OWASP — Double encoding / input validation bypass background — https://owasp.org/www-community/Double_Encoding
- FIRST.org — CVSS 3.1 Specification Document — https://www.first.org/cvss/v3-1/specification-document

Appendix A: Evidence Index

All screenshots in this report were captured directly from Burp Suite's Proxy/Repeater history during the live engagement, and are reproduced faithfully (request and response bytes unmodified) for evidentiary purposes.

Figure	Description
Fig. 1	VULN-01 — Baseline request (GET /image?filename=54.jpg) confirms filesystem-backed file serving
Fig. 2	VULN-01 — Bypass payload (....//....//....//etc/passwd) returns the full contents of /etc/passwd
Fig. 3	VULN-02 — Baseline request (GET /image?filename=7.jpg) confirms filesystem-backed file serving on the second instance
Fig. 4	VULN-02 — Direct traversal payload (../../../etc/passwd) returns the full contents of /etc/passwd with zero filtering
Fig. 5	VULN-03 — Baseline request (GET /image?filename=23.jpg) confirms filesystem-backed file serving on the third instance
Fig. 6	VULN-03 — Double URL-encoded payload (..%252f..%252f..%252fetc/passwd) returns the full contents of /etc/passwd
Fig. 7	VULN-04 — Baseline request (GET /image?filename=/var/www/images/44.jpg) confirms absolute-path acceptance within the allowed base directory
Fig. 8	VULN-04 — Prefix-plus-traversal payload (/var/www/images/../../../etc/passwd) returns the full contents of /etc/passwd
Fig. 9	VULN-05 — Baseline request (GET /image?filename=29.jpg) confirms filesystem-backed file serving on the fifth instance
Fig. 10	VULN-05 — Null-byte payload (../../../etc/passwd%00.png) returns the full contents of /etc/passwd despite a Content-Type: image/png header

Appendix B: Tools & Techniques Used

Tool	Purpose
Burp Suite (Proxy/Repeater)	Traffic inspection and crafting/replaying all five path-traversal payloads against the five target endpoints
Manual payload analysis	Working through the non-recursive single-pass replace() logic (VULN-01), the decode-pipeline ordering flaw (VULN-03), the prefix-only validation gap (VULN-04), and the null-byte string-termination mismatch (VULN-05) to derive each bypass sequence

Appendix C: Disclaimer & Legal Notice

This report documents the results of security testing performed against a designated training-range target as part of an authorized security exercise, conducted under the scope and rules of engagement described in Section 2. Findings, payloads, extracted data, and screenshots are provided solely for educational and defensive remediation purposes.

This assessment represents a point-in-time evaluation of the target's security posture and is based on the information and access available during the stated testing window. It does not constitute a guarantee that no other vulnerabilities exist, nor that the target will remain secure against future threats. The techniques, tools, and payloads described in this report must not be used against any system without explicit, documented authorization from its owner. The authors accept no liability for any unauthorized or illegal use of the information contained herein.

All trademarks, product names, and company names referenced in this report are the property of their respective owners and are used for identification purposes only.